Predicting Pokémon Battles by Rebuilding What the Log Hides

Matteo Angeloro Lorenzo Amoruso Carmine Rauso Lorenzo Valenti
Sapienza University of Rome
{angeloro.1931328,amoruso.2157399,rauso.2341114,valenti.2102514}@studenti.uniroma1.it

Abstract

We predict whether Player 1 wins a Gen 2 OU Pokémon battle from its first 30 recorded turns. The key observation is that the battle log hides most of Player 2's knockouts: a fainted Player 2 Pokémon looks like an ordinary switch. Using Player 1's always-visible knockouts as a control group, a classifier estimates which Player 2 switches were knockouts, and the corrected game state feeds a LightGBM model together with an out-of-fold encoding of Player 1's team history. The model reaches 0.699 accuracy in repeated cross-validation, 0.700 on the public leaderboard and 0.683 on the private one.

1. Introduction

We are given 10,000 labelled and 5,000 test Gen 2 OU battles. From Player 1's team, Player 2's lead and the first 30 recorded turns we must say whether Player 1 wins.

What does the log really show? Two facts emerged. Player 2's `fnt` state appears almost only in double knockouts: about one Player 2 knockout per battle looks like an ordinary switch. And the same Player 1 team recurs across battles, with a win rate that predicts the outcome, while Player 2's does not. How can we turn these facts into features?

2. Evaluated Approaches

Feature engineering. The state at turn 30 taken at face value (Pokémon alive, HP, statuses, active Pokémon, boosts, Spikes) scored 0.675. Adding Player 1's species and items and the difference of move counts between the players gave a baseline of 0.685.

This state still counts Player 2's hidden knockouts as alive. How can we find them? Player 1's knockouts are always visible, so every Player 1 Pokémon that leaves and never comes back (a "vanishing") is a real switch. If both players switch alike, the extra vanishings of Player 2 are the hidden knockouts. A LightGBM classifier separates Player 2's vanishings from Player 1's using only the event (HP before leaving, whether the replacement entered unhurt, move powers). With q its output, $(1-q)/q$ is the share of real switches among Player 2's events, so $P(\mathrm{KO} \mid x) = 1-c\,(1-q)/q$, with c matching the observed excess. Without labels, Player 2 now loses as many Pokémon as Player 1 (1.61 vs. 1.69 per battle). We recompute the state weighting each Pokémon by its probability of being alive.

What about Player 1's identity? We add the win rate of Player 1's exact team, target-encoded out-of-fold inside each training fold.

Models. Our learner is LightGBM with shallow trees; logistic regression was 0.5 points worse, and XGBoost, CatBoost and blends stayed within ± 0.003.

An alternative pipeline. Could we instead forecast what happens after turn 30? A second pipeline first trains turn-level models that predict, for each player, the next switch and the type of the next active Pokémon, and turns them into expected matchups at turn 31. With Gen 2 damage estimates and metagame frequencies from the training teams (about 120 features), it feeds gradient-boosting ensembles (v2 a stack, v5 a CatBoost-dominant blend).

Validation. With differences this small, how do we know a block helps? All configurations share one stratified 5-fold CV repeated three times, compared by paired fold differences with the Nadeau–Bengio corrected standard error [1].

3. Results

Table 1. Our three final submissions (accuracy).

Submission	Validation	Public	Private
Hidden knockouts + history	**0.699**	**0.700**	**0.683**
Turn-31 forecast, v5	0.689	0.697	0.683
Turn-31 forecast, v2	0.681	0.695	0.675

Against the 0.685 baseline, our final model gains $+0.014 \pm 0.004$, almost all from the hidden-knockout correction ($+0.012 \pm 0.003$); the history adds only $+0.002 \pm 0.004$. On the private leaderboard (Table 1) the two best tie at 0.683, 1.5 points below validation. The forecast pipeline tuned its blend on the same out-of-fold predictions, so its validation is optimistic. We prefer the hidden-knockout model because its gain is explained. Code at https://github.com/teoang/fds-pokemon-challenge.

References

[1] Claude Nadeau and Yoshua Bengio. Inference for the generalization error. *Machine Learning*, 52(3):239–281, 2003. 1